# FPGA Beamformer

Stewart Nash

2026

# Chapter 1

# Realization of Digital Linear Systems

The realization of discrete systems is accomplished by converting the system transfer function into an algorithm for implementation via a digital network. The implementation of the algorithm represents the selected form of the difference equation considering computational requirements and finite word-length effects on system performance. The process of determining the optimum realization for the digital network is iterative and is initiated in step 3, *Signal Prcoessing Design*, of the system design methodology. The resources of the signal processor hardware are analyzed for the proposed implementaion in step 4, *Resource Analysis*. When an acceptable design is achieved, the iteration is stopped. In this chapter we introduce basic digital linear system realizations and show the use of the Z-transform in the realization analysis. The iterative analysis considering finite wordlength effects and computational requirements will be left to Chapters 9 and 10.

## 1.1 Network Definitions and Operations

In Chapter 2 we introduce three basic operations – addition, multiplication, and unit delay – that are used to implement the systems described here. In addition, sampling rate decrease, sampling rate increase, and modulation operations were defined. We now consider the general solution of the response of the system, $y(nT)$, in terms of the input and the network. In Section 3.4.1 we developed the general $N$th-order system transfer function given by Eq. 3.36, that is, the system response function can be written as

$$Y(z) = X(z)H(z) \tag{1.1}$$

Thus, the Z-transform of the response $Y(z)$ is determined as the product of the Z-transform of the excitation function $X(z)$ and the transfer function $H(z)$. We have shown that the response of the system can be efficiently determined using $z$-domain analysis.

The *transpose* of a network is defined by the following operations:

- Reverse the direction of all branches in the signal flow graph.
- Interchange the inputs and outputs.
- Reverse the roles of all nodes in the flow graph.
  - Summing points become branching points.
  - Branching points become summing points.

Linear time-invariant branch operations remain unchanged in the process of transposition, where only the direction of the branches are reversed. Finally, the system transfer function is unchanged by transposition.
Figure 3.7a is the transpose of the network shown in Fig. 3.6b showing the input on the right and the output on the left. The figure is redrawn in Fig. 3.7b in the normal form with the input on the left. It will be shown in Chapter 7 that the transpose operation provides a method for obtaining filter networks that allow more efficient multirate implementations. Also, in Chapter 9 we show that hte network implementation affets the errors that result from finite arithmetic effects.
The dual of a network performs a complementary function to the original network. For example, modulators and demodulators are dual operations. Given a network, its dual is found by transposing the network. Any linear time-invariant or time-varying network has a dual. Interpolators and decimators are dual systems. In Chapter 7 we will see that the dual of a decimator (interpolator) will often result in an efficient form for an interpolator (decimator).
Commutation is the changing of the order of the network operations, as shown in Fig. 3.8 without changing the network results. This can provide for computational savings, as shown in the figure by performing the multiplication prior to the branch operation.

## 1.2 IIR Direct-Form Filter Realizations

The structure of Figure 3.6a is referred to as a direct form 1 realization of a second-order IIR filter. Note that the feed-forward portion is implemented

first as followed by the feedback plrtion. Figure 3.6b defines director form 2 realization, which is canonic (i.e., the number of storage delays equals the order of the filter when implemented with common delay elements). This structure implements the feedback portion prior to the feed-forward section. The digital network for implementing the general difference equation of Eq. 3.35 is shown in Figure 3.9a. This network is a direct form 1 realization of the system characterized by Eq. 3.36. We have used an alternative form of network representation where the multiplications are represented by the arrows and summing nodes are given by any node with two or more branch inputs. A direct form implementation is obtained by converting the system transfer function into the corresponding difference equation and using our basic operations to implement the difference equations directly. Note that this implementation requires $N$ unit delays for both the feedback portion and the feed-forward portion of the network. Each unit delay represents a requirement for a register to store the data. The multiplications and additions must be performed at the input sample rate. Figure 3.9b is the corresponding direct form 2 IIR filter implementation.
It has been shown, that when $N$ is large the coefficient accuracy required to iumplement the network and the arithmetic implementation requirements are high. Leland Jackson also analyzed several alternative form digital filter implementations and showed that parallel and cascade realizations result in lower arithmetic requirements than does the direct form.

## 1.3 IIR Cascade and Parallel Filter Realizations

It can be seen from Figure 3.9a that the realization of $H(z)$ was factored into the realization of two transfer functions, that is

$$H(z) = H_D(z)H_N(z) = \left(\frac{1}{\sum_{i=0}^{N} B_i z^{-i}}\right)\left(\sum_{i=0}^{N} A_i z^{-i}\right) \tag{1.2}$$

Equation 3.59 gives $H(z)$ as the product of the denominator portion of the transfer function, $H_D(z)$, with the numerator portion given by $H_N(z)$. This is referred to as a cascade factorization of the transfer function into the feed-forward portion resulting from the zeros (the numerator) and the feedback portion resulting from the polls (the denominator). Each of these portions is shown in Figure 3.9.
We can reverse the order (i.e., commutation rule) of the cascade operations without changing the transfer function. The direct form is shown in Figure 3.9a with the numerator and denominator terms commuted. Since the unit

delays (storage elements) for both portions contain identical data, the delays can be combined, thereby reducing the storage required for implementing the filter. Thus Figure 3.9b requires only one half the number of unit delay operations as required by Figure 3.9a. The resulting digital network is called direct form 2 or canonic form, that is, the number of delays equal the filter order.

It can be seen from Figure 3.9 that if instant $nT$ is takne to be the present, the present response of an IIR filter is a function of the present and past $N$ values of the input as well as the past values of the response. If we let the coefficients $B_i = 0$, then Eq. 3.60 simplifies to a FIR digital filter. For this case the output samples of the system $y(nT)$ depend only on the present and past input samples.

Figures 3.10a and b illustrate a sixth-order filter realization by three second order sections in cascade and parallel, respectively. The coefficients for the cascade or parallel realization are obtained by factoring or partial fraction expansion of the direct-form transfer function, $H(z)$.

# Chapter 2

# Lattice Filters

## 2.1 Single-Multiplier All-Pass Filter

### 2.1.1 Introduction

All-pass filters alter the phase of a signal while preserving its magnitude spectrum: the all-pass filter should ideally have unity gain at all frequencies. The purpose of an all-pass filter is thus temporal and phase manipulation. Therefore, all-pass filters are used in applications such phase equalization, group-delay compensation, quadrature signal generation, reverberation systems, filter banks, and adaptive signal processing.
In many DSP systems, particular embedded systems and real-time hardware, multipliers dominate the computational cost. Considerable research has gone into reducing the number of multipliers used to realize a digital filter. A single-multiplier all-pass filter allows one to achieve a higher-order phase response without arithmetic complexity.

### 2.1.2 General Theory

The magnitude response of a digital all-pass filter is

$$|H(e^{j\omega})| = 1 \tag{2.1}$$

If the phase response is $\phi(\omega)$, the group delay is given by

$$\tau_g(\omega) = -\frac{d\phi(\omega)}{d\omega} \tag{2.2}$$

A first-order digital all-pass filter has the following transfer function

$$H(z) = \frac{\alpha + z^{-1}}{1 + \alpha z^{-1}} \tag{2.3}$$

where $|\alpha| < 1$ for stability. The transfer function of a second-order section may be written as

$$H(z) = \frac{\alpha_0 + \alpha_1 z^{-1} + z^{-2}}{1 + \alpha_1 z^{-1} + \alpha_0 z^{-2}} \tag{2.4}$$

In an all-pass filter, the poles and zeros are reciprocal conjugates. In other words, if a pole is at $p$, the corresponding zero will be at $(p^*)^{-1}$. So, every pole inside the unit circle will have a corresponding zero reflected outside the unit circle. This symmetry is what allows for a unity magnitude response.

Higher order all-pass filters are typically constructed by cascading first and second order sections. And a traditional implementation would use a direct-form or cascade realization. The second order filter given by the transfer function above generally requires several multipliers, including one multiplier per coefficient, multiple additions and delay elements. For an $N$-th order filter, the arithmetic cost scales roughly with $N$. Multipliers consume substantially more silicon area and power than adders or delay elements. THis was problematic in early DPS chips, FPGA realizations, ASIC implementations, and low-power embedded systems.

A multi-multiplier all-pass section may be realized in lattice form, wave digital form, coupled all-pass form, and state-space canonical form. The computational realization differs drastically between these forms, though they represent the same transfer function.

### 2.1.3 Single-Multiplier Justification

Many all-pass structures can be written in a lattice or coupled form using only one coefficient multiplication per stage. For example, a common first order filter with one coefficient is given by

$$y[n] = -ax[n] + x[n-1] + ay[n-1] \tag{2.5}$$

Single-multiplier structures benefit from numerical robustness. Direct-form recursive filters are sensitive to coefficient quantization because poles may shift outside of the unit circle. However, lattice all-pass filters only require that the coefficient magnitude is less than 1.

## 2.2 All-Pass Filter

$$\begin{aligned} w_1[n] &= w_4[n] - x[n] \\ w_2[n] &= \alpha w_1[n] \\ w_3[n] &= w_2[n] + x[n] \\ w_4[n] &= w_3[n-1] \\ y[n] &= w_2[n] + w_4[n] \end{aligned} \tag{2.6}$$

$$Y(z) = \frac{z^{-1} - \alpha}{1 - \alpha z^{-1}} X(z) \tag{2.7}$$

$$H(z) = \frac{z^{-1} - \alpha}{1 - \alpha z^{-1}} \tag{2.8}$$

## 2.3 M-Order Filter

$m$-degree polynomial

$$a_n(n) = K_n \tag{2.9}$$

$$a_{n-1}(i) = \frac{a_n(i) - a_n(n) - a_n(n-1)}{1 - a_n^2(n)} \tag{2.10}$$

# Chapter 3

# Adaptive Filtering

*"Filters that adapt to a changing environment are discussed. The stochastic Wiener filtering and deterministic least-squares problems are set up, and the similarity between them is pointed out. First, a block-processing approach is taken in the solution to these two problems. Second, a fading memory (recursive) approach is taken. Then the application of these algorithms to the adaptive beamforming (ABF) problem is considered. The chapter appendix provides background material on conventional beamforming (CBF)."*

## 3.0 Introduction

Numerous applications exist that require a linear filtering operation. Often, the nature of that filtering task is time-varying in some nondeterministic fashion owing to nonstationarity of the underlying time series. In such situations, a filter that can adapt to a changing environment is needed. Examples include adaptive noise canceling, line enhancing, frequency tracking, and channel equalization. Beyond the discussion here, additional material on adaptive filtering can be found in references 1 through 16.

## 3.1 The Stochastic Wiener Filtering and Deterministic Least Squares Problem

The problem of interest is described in Figure 11.1. The goal is to filter the time series $x[n]$ with a causal, finite impulse response (FIR) digital filter

to yield an estimate of the time series $s[n]$. The error in that estimate is denoted $e[n]$

$$e[n] = s[n] - \hat{s}[n] = s[n] + \sum_{k=0}^{N} h_k x[n-k] \tag{3.1}$$

Written in matrix form, Eq. 11.1 becomes

$$e[n] = s[n] + [h_0\, h_1\, \ldots\, h_N] \begin{bmatrix} x[n] \\ x[n-1] \\ \vdots \\ x[n-N] \end{bmatrix} \tag{3.2}$$

or, in vector notation,

$$e[n] = s[n] + \mathbf{h}^T \mathbf{x} \tag{3.3}$$

The solution to this problem for the unknown filter vector $\mathbf{h}$ can be approached in two ways, depending on the optimality criterion chosen

$$\min_{\mathbf{h}} E\left[e^2[n]\right] \tag{3.4}$$

*(Wiener filtering problem)*

$$\min_{\mathbf{h}} \sum |e[n]|^2 \tag{3.5}$$

*(Least squares problem)* The Wiener filtering problem takes a stochastic viewpoint where the filter is optimized in an expected value sense (minimum mean-squared error). As an alternative, the least-squares problem views the time series involved as deterministic and the filter is optimized for the specific sequences of record (minimum sum-squared error). Both viewpoints give rise to similar mathematical expressions for $\mathbf{h}$.

The distinction between the Wiener filtering and least squares problems is made in the definitions of $R_{ss}[0]$, $\mathbf{v}$ and $\mathbf{T}$ where

$$\mathbf{v} = \begin{bmatrix} R_{xs}[0] \\ R_{xs}[1] \\ \vdots \\ R_{xs}[N] \end{bmatrix} \tag{3.6}$$

$$\mathbf{T} = \begin{bmatrix} R_{xx}[0] & R^*_{xx}[1] & \cdots & R^*_{xx}[N-1] & R^*_{xx}[N] \\ R_{xx}[1] & R_{xx}[0] & \cdots & R^*_{xx}[N-2] & R^*_{xx}[N-1] \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ R_{xx}[N-1] & R_{xx}[N-2] & \cdots & R_{xx}[0] & R^*_{xx}[1] \\ R_{xx}[N] & R_{xx}[N-1] & \cdots & R_{xx}[1] & R_{xx}[0] \end{bmatrix} \tag{3.7}$$

For the Wiener filtering problem

$$R_{ss}[0] = E[s[n]s^*[n]] \tag{3.8}$$

$$\mathbf{v} = \{E[s[n]x^*[n-j]]\} = \{R_{sx}[j]\} \tag{3.9}$$

$$\mathbf{T} = \{E[x[n-k]x^*[n-j]]\} \tag{3.10}$$

And, for the least squares problem

$$R_{ss}[0] = \sum_n s[n]s^*[n] \tag{3.11}$$

$$\mathbf{v} = \{\sum_n s[n]x^*[n-j]\} = \{v_j\} \tag{3.12}$$

$$\mathbf{T} = \{\sum_n x[n-k]x^*[n-j]\} \tag{3.13}$$

The next step in solving for the unknown filter vector $\mathbf{h}$ is to write an expression for squared error based on Eq. 11.3

$$|e[n]|^2 = s^*[n]s[n] + s^*[n]\mathbf{h}^T\mathbf{x} + \mathbf{h}^\dagger\mathbf{x}^*s[n] + \mathbf{h}^\dagger\mathbf{x}^*\mathbf{h}^T\mathbf{x} \tag{3.14}$$

By making use of the definitions of $R_{ss}[0]$, $\mathbf{v}$, and $\mathbf{T}$ the average squared error $\varepsilon(\mathbf{h})$ can be derived from Eq. 11.14 for both the Wiener filtering and least squares problems as

$$\varepsilon_N(\mathbf{h}) = R_{ss}[0] + \mathbf{v}^\dagger\mathbf{h} + \mathbf{h}^\dagger\mathbf{v} + \mathbf{h}^\dagger\mathbf{T}^T\mathbf{h} \tag{3.15}$$

Now, minimizing $\varepsilon(\mathbf{h})$ with respect to $\mathbf{h}$ leads to the following equation

$$0 = \mathbf{v} + \mathbf{T}^T\mathbf{h} \tag{3.16}$$

or

$$\mathbf{T}^T\mathbf{h} = -\mathbf{v} \tag{3.17}$$

Expressing Eq. 11.17 in expanded form wil be of interest in the next section

$$\begin{bmatrix} R_{xx}[0] & R_{xx}^*[1] & \cdots & R_{xx}^*[N-1] & R_{xx}^*[N] \\ R_{xx}[1] & R_{xx}[0] & \cdots & R_{xx}^*[N-2] & R_{xx}^*[N-1] \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ R_{xx}[N-1] & R_{xx}[N-2] & \cdots & R_{xx}[0] & R_{xx}^*[1] \\ R_{xx}[N] & R_{xx}[N-1] & \cdots & R_{xx}[1] & R_{xx}[0] \end{bmatrix} \begin{bmatrix} h_0 \\ h_1 \\ \vdots \\ h_{N-1} \\ h_N \end{bmatrix} =$$

$$- \begin{bmatrix} R_{xs}[0] \\ R_{xs}[1] \\ \vdots \\ R_{xs}[N-1] \\ R_{xs}[N] \end{bmatrix} \tag{3.18}$$

or

$$\sum_{j=0}^{N} R_{xx}[j-i]h_j = -R_{xs}[j] \quad i = 0, 1, \ldots, N \tag{3.19}$$

Now, solving Eq. 11.17 for $\mathbf{h}$ yields

$$\mathbf{h} = -(\mathbf{T}^T)^{-1}\mathbf{v} \tag{3.20}$$

## 3.2 Block Processing

One approach to processing time series is to split an observation interval into subintervals. The subinterval (sample collection) is then processed as a block.

## 3.3 Fading Memory

An adaptive filter can be continuously updated, and a fading memory of past data can be incorporated into its algorithm.

### 3.3.1 Joint Complex Gradient Transversal Filter

[JCLMS] A joint complex gradient transversal filter (JCGTF or JCLMS transversal filter) can be conceived of as a complex-valued LMS adaptive FIR filter. The filter output is given as

$$y(n) = \mathbf{w}^{\dagger}(n)\mathbf{x}(n) \tag{3.21}$$

where $\mathbf{x}(n)$ is the complex input vector, $\mathbf{w}(n)$ is the complex coefficient vector and the dagger represents the Hermitian transpose. The error is given by

$$e(n) = d(n) - y(n) \tag{3.22}$$

and the coefficient update is

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu\mathbf{x}(n)e^*(n) \tag{3.23}$$

where $d(n)$ is the desired signal, $\mu$ is the adaptation step size, and the asterisk denotes complex conjugation.

### Widely linear (Augmented) JCLMS

The augmented complex LMS is given by

$$y(n) = \mathbf{w}^{\dagger}\mathbf{x}(n) + \mathbf{g}^{\dagger}\mathbf{x}^{*}(n) \tag{3.24}$$

for non-circular complex signals. The update equations are then

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu\mathbf{x}(n)e^{*}(n) \tag{3.25}$$

$$\mathbf{g}(n+1) = \mathbf{g}(n) + \mu\mathbf{x}^{*}(n)e^{*}(n) \tag{3.26}$$

## 3.4 Joint Complex Gradient Lattice Filter

[JCGL] A Joint Complex Gradient Lattice (JCGL) filter is the lattice counterpart of the JCLMS transversal filter. Rather than adapting FIR tap weights directly, it adapts a set of forward and backward prediction errors and reflection coefficients (PARCOR coefficients).

For an order-$M$ complex lattice, define the forward prediction error as

$$f_m(n) = f_{m-1}(n) - k_m(n)b_{m-1}(n-1) \quad , f_0(n) = x(n) \tag{3.27}$$

and backward prediction error

$$b_m(n) = b_{m-1}(n-1) - k_m^{*}(n)f_{m-1}(n) \quad , b_0(n) = x(n) \tag{3.28}$$

The reflection coefficients $k_m$ are adapted by a complex gradient rule. A commonly used JCGL adaptation is

$$k_m(n+1) = k_m(n) + \mu f_m(n)b_m^{*}(n) \tag{3.29}$$

where $\mu$ is the adaptation step size.

### Joint Complex Gradient Lattice Predictor

Many texts define the JCGL as an adaptive predictor rather than merely a lattice analyzer. In that case, the predictor output is

$$\hat{x}(n) = -\sum_{m=1}^{M} a_m(n)x(n-m) \tag{3.30}$$

and the final-stage forward error

$$e(n) = f_M(n) \tag{3.31}$$

becomes the prediction error.

**Joint Complex Gradient Lattice-Ladder**

[JCGLL] Many systems extend the lattice with a ladder section

$$y(n) = \sum_{m=0}^{M} v_m(n) b_m(n) \tag{3.32}$$

where the lattice coefficients $k_m$ perform whitening and ladder coefficients $v_m$ perform system identification. This structure is the complex analog of the JCLSL (Joint Complex Least Squares Lattice).

## 3.5 Joint Complex Least Squares Lattice Filter

[JCLSL] A joint complex least squares lattice (JCLSL) filter combines

1. A complex lattice predictor (forward/backward prediction errors)
2. A least-squares ladder section
3. Recursive estimation of reflection coefficients and ladder weights

It can be conceived of as the complex-valued adaptive lattice equivalent of an RLS filter. The structure of the filter has the input $x(n)$ go through a lattice and have $b_m$ applied and go through a ladder to produce an output $y(n)$. The final output is given by

$$y(n) = \sum_{m=0}^{M} v_m(n) b_m(n) \tag{3.33}$$

where $v_m$ are the ladder coefficients.

# Chapter 4

# Wavelet Analysis

## 4.1 Introduction

This supplemental chapter is an introduction to wavelet theory.

## 4.2 Wavelets

A wavelet is a transient oscillation. Wavelets can be used to analyze data by creating a wavelet series representation of a square-integrable function with respect to either a complete, orthonormal set of basis functions, or an overcomplete set or frame of a vector space, for the Hilbert space of square-integrable functions. (This is accomplished through coherent states.)

A wavelet transform may be considered a form of time-frequency representation for analog signals. A wavelet approximation can be made with a discrete wavelet transform of a sampled signal by using discrete-time filter-banks in a dyadic configuration.

## 4.3 Wavelet Transform

A wavelet series is a representation of a square-integrable function by a certain orthonormal series generated by a wavelet. A function $\psi \in L^2(\mathbb{R})$ is called an orthonormal wavelet if it can be used to define a Hilbert basis. The Hilbert basis is constructed as the family of functions $\{psi_{jk} : j, k \in \mathbb{Z}\}$ by means of dyadic translations and dilations of $\psi$,

$$\psi_{jk}(x) = 2^{j/2}\psi(2^j x - k) \tag{4.1}$$

for integers $j, k \in \mathbb{Z}$. All functions $f \in L^2(\mathbb{R})$ may be expanded in the basis a a wavelet series

$$f(x) = \sum_{j,k=-\infty}^{\infty} c_{jk}\psi_{jk}(x) \tag{4.2}$$

The integral wavelet transform is defined as

$$[W_\psi f](a,b) = \frac{1}{\sqrt{|a|}} \int_{-\infty}^{\infty} \psi\left(\frac{\bar{x} - b}{a}\right) f(x)\, dx \tag{4.3}$$

The wavelet coefficients $c_{jk}$ are then given by

$$c_{jk} = [W_\psi f](2^{-j}, k2^{-j}) \tag{4.4}$$

Here, $a = 2^{-j}$ is called the binary dilation or dyadic dilation, and $b = k2^{-j}$ is the binary or dyadic position.